PRACTICAL 15

StudentHub Portal – Final Integrated Submission

Student Name	KIRTAN KAKADIYA
Enrollment / Roll No.	25CS026
Program / Semester	3RD
Course / Subject	WEB DEVELOPMENT FRAMEWORK
Faculty Name	BRINDA MAM
Submission Date	31-10-2026

1. Aim

To integrate the earlier practicals into one StudentHub academic portal (public pages, PHP login with Admin and Student roles, student and event administration, JSON API, dashboard and audit logs), and to document its setup, testing, security and accessibility for final submission.

2. Submission Checklist

Item	Status	Deliverable
GitHub repository	Source code pushed to a **private** repository	https://github.com/25cs026-oss/WDF-ITUE203
Database dump	Ready – schema and fictional starter records only	`studenthub.sql`
README.md	Ready – XAMPP setup, import, demo logins and testing steps	`README.md`
Test & security report	Ready – sections 3, 4 and 5 of this document	This report
Screenshots / demo	Ready – integrated UI and authentication proof	Section 6

3. Functional Test Summary

Area	Result
Student account administration	Pass – table, create, update, search, course filter, soft delete, messages and CSV export
Event administration	Pass – create/update/delete, poster display, invalid type and size handling, Student denial
Student JSON API (Fetch)	Pass – list/create/update/delete, validation, error and loading states, Fetch status evidence
Dashboard and role-specific navigation	Pass – live user/student/event counts, latest registrations, Student menu
Audit logging	Pass – student create/update/deactivate and event update entries retained after the temporary test account was removed
Admin-only route protection	Pass – Student session received HTTP 403 from `admin_dashboard.php`, `event_management.php` and `audit_logs.php`

4. Security Checklist

Control	Result / evidence
Password handling	Hashes verified with `password_verify()`; demo passwords are stored as hashes in the SQL dump.
Session management	Strict session mode, HttpOnly and SameSite=Lax cookies, session ID regeneration on login, 15-minute inactivity timeout.
Role-based authorization	Admin routes check the role on the server; Student attempts returned HTTP 403. Hiding a link is not the authorization boundary.
CSRF defenses	Session-bound tokens checked for student/event writes, API write requests and logout.
Database access	User-controlled values in the reviewed CRUD/dashboard flows use prepared MySQLi statements.
Output encoding	Dynamic PHP-rendered user, event and audit values are HTML-escaped.
Poster uploads	MIME type and image contents checked; JPEG/PNG/WebP only; 2 MB limit; random file names; script-like extensions denied in the upload folder. Invalid type and oversize tests were rejected.
Audit integrity	Successful student/event changes and their audit records are committed in the same database transaction.
SQL dump privacy	The dump excludes live development account rows and test audit history; only fictional demo data is included.
External security assessment	Not performed. This is a practical-level checklist, not a penetration test or a production-certification claim.

5. Accessibility Checklist

Check	Result
Page titles, primary headings, semantic main/navigation landmarks	Checked on the integrated login and dashboard views.
Form labels; readable names on buttons and links	Checked in the PHP forms and captured role views.
Tables have header cells and responsive overflow containers	Checked in student, event, dashboard and audit table views.
Status/error content readable without relying on colour alone	Checked in the tested success/error flows.
Uploaded posters have descriptive alternative text	Checked in the event-list template.
Narrow-screen layout (390 px)	Checked for home, login, Admin dashboard and Student Records; no document-level horizontal overflow observed.
Automated WCAG/axe scan, measured contrast, assistive-technology test	Not performed; recommended before making a formal WCAG conformance claim.

6. Screenshots – Integrated UI and Authentication Flow

6.1 Integrated public home page

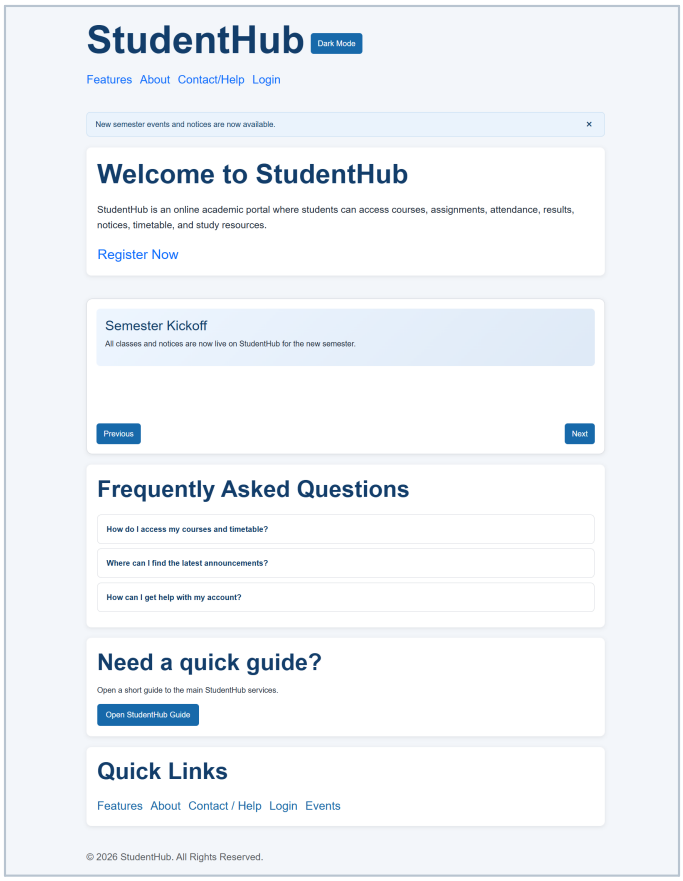


Figure 6.1: Integrated public home page

6.2 Login page

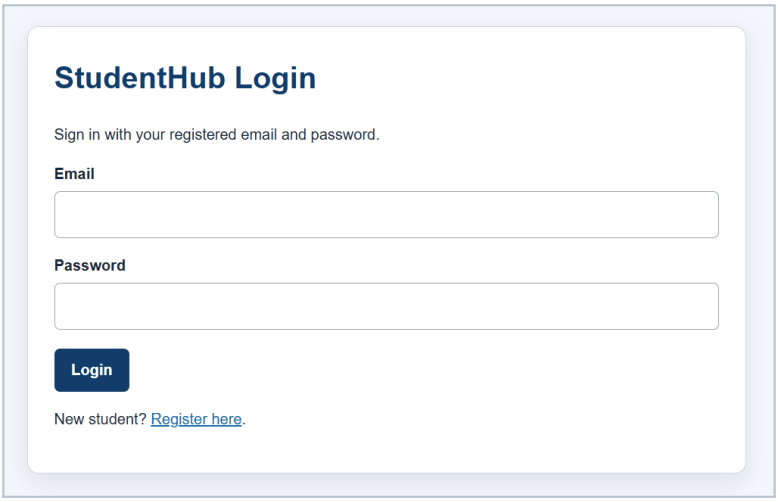


Figure 6.2: Login page

6.3 Admin dashboard after login (Admin role menu, live statistics)

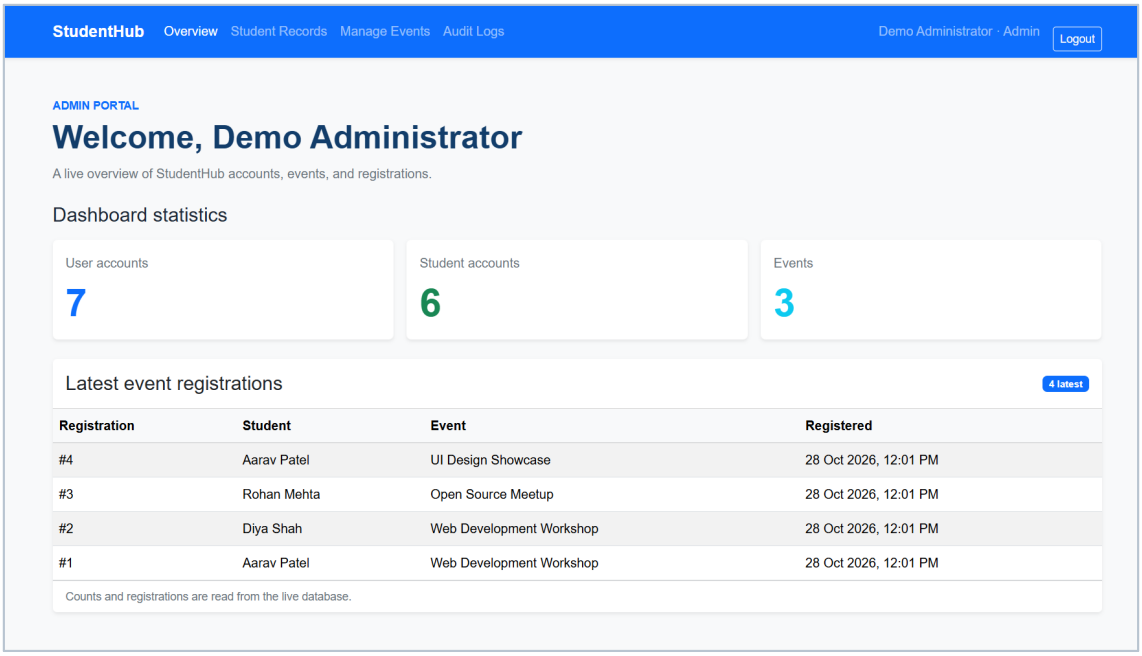


Figure 6.3: Admin dashboard after login (Admin role menu, live statistics)

6.4 Student dashboard after login (Student role menu)

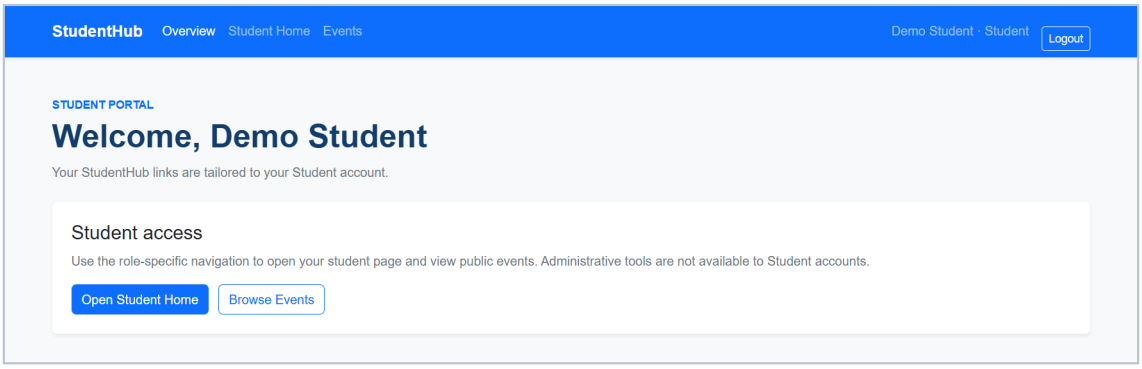


Figure 6.4: Student dashboard after login (Student role menu)

7. Setup and Testing Instructions

Full steps are in `README.md`. In short:

- Start **Apache** and **MySQL** in XAMPP (Live Server alone does not run PHP).
- Back up any existing StudentHub database, then import `studenthub.sql` using phpMyAdmin or the MySQL command line. The dump drops and recreates the project tables and contains only fictional demo data; the audit log and Fetch API tables start empty.
- Open `http://localhost/studentHUB/home.html` for the public UI, or `http://localhost/studentHUB/login.php` for PHP login.
- Sign in with the fictional demo accounts below and test each role.

Role	Email	Password
Admin	admin.demo@example.test	AdminDemo9!
Student	student.demo@example.test	StudentDemo9!

These are fictional local-test credentials; do not reuse them outside the demonstration environment.

8. Deployment Follow-ups and Limitations

- Serve the app over HTTPS in production; the local XAMPP demonstration uses HTTP.
- Replace the empty-password local MySQL defaults with a dedicated database user and strong secrets.
- Review production security headers, especially a Content Security Policy compatible with the Bootstrap CDN.
- Change or remove the demo credentials before any deployment, maintain backups and use least-privilege database permissions.

9. Result

The StudentHub portal was integrated and tested end to end. The public pages, PHP authentication with Admin and Student roles, student and event administration, Fetch-based JSON API, live dashboard and audit logging all work as documented, and Admin-only routes return HTTP 403 to Student accounts. The source code, database dump, README and test evidence are ready for submission.

10. Conclusion

This final practical combined front-end design, PHP and MySQL back-end work, role-based access control, secure file handling and audit logging into one project. Testing and the security and accessibility checklists show a sound practical-level implementation, and the listed follow-ups describe what would still be needed before any production use.

— End of Practical 15 Submission —